

Session 10 — Bonds --- Treasury curve and OECD 10Y

In class (90 min). Type along with the instructor. The project is **not graded**. The final exam is a closed-book MCQ: you must be able to **explain every line** you type.

Goal. Load FRED series with no API key and draw four yield charts.

You leave with. The Bonds tab complete. This finishes the **minimum path** (with Stocks).

```
uv run streamlit run dashboard.py
```

Yields are not Yahoo prices. We use **FRED** through `pandas_datareader (DataReader(..., "fred"))`. No FRED API key.

If the network blocks FRED, bundled files `data/DGS*.csv` and `data/IRLTLT01*.csv` still plot. Same idea as `GSPC.csv`.

FRED download in `views/common.py`

Add import `pandas_datareader.data` as `web` at the top of `common.py`.

Reuse `_live_fetch_start` and `_merge_live_and_stored` (FRED ids are filenames as-is).

```
@st.cache_data(show_spinner=False, ttl=60 * 60)
def _fetch_live_fred(
    series_ids: Tuple[str, ...],
    start_date: date,
    end_date: date,
) -> pd.DataFrame:
    if not series_ids:
        return pd.DataFrame()
    try:
        frame = web.DataReader(list(series_ids), "fred", start_date, end_date)
    except Exception:
        return pd.DataFrame()
    if frame is None or frame.empty:
        return pd.DataFrame()
    if isinstance(frame, pd.Series):
        frame = frame.to_frame(name=series_ids[0])
    frame.index = pd.to_datetime(frame.index).tz_localize(None)
    return frame

def download_fred_data(
    series_ids: Tuple[str, ...],
    start_date: date,
    end_date: date,
    *,
    use_live: bool = True,
) -> pd.DataFrame:
    if not series_ids:
        return pd.DataFrame()
    series_ids = tuple(series_ids)
    live_start = _live_fetch_start(series_ids, start_date, end_date) if use_live else None
    live_frame = (
        _fetch_live_fred(series_ids, live_start, end_date)
        if live_start is not None
        else pd.DataFrame()
    )
    merged = []
    for series_id in series_ids:
        live_series = live_frame[series_id] if series_id in live_frame.columns else None
        if live_series is not None:
            live_series = pd.to_numeric(live_series, errors="coerce").rename(series_id)
        series = _merge_live_and_stored(series_id, live_series)
```

```

        if not series.empty:
            merged.append(series)
    if not merged:
        return pd.DataFrame()
    fetched = pd.concat(merged, axis=1).sort_index()
    mask = (fetched.index >= pd.Timestamp(start_date)) & (
        fetched.index <= pd.Timestamp(end_date)
    )
    return fetched.loc[mask]

```

Series catalogues in views/bonds.py

Type the FRED ids with the instructor (Treasury tenors + OECD 10-year names). Keep bonds_ prefixes on **all** sidebar keys.

```

from datetime import date

import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from views.common import (
    DEFAULT_END_DATE,
    LOOKBACK_YEARS,
    chart_layout,
    date_range_error,
    download_fred_data,
    get_available_date_bounds,
    lookback_start,
)

CHART_COLOR = "#FF962F"

TREASURY_SERIES = [
    ("DGS1M0", "1M", 1 / 12),
    ("DGS3M0", "3M", 0.25),
    ("DGS6M0", "6M", 0.5),
    ("DGS1", "1Y", 1),
    ("DGS2", "2Y", 2),
    ("DGS3", "3Y", 3),
    ("DGS5", "5Y", 5),
    ("DGS7", "7Y", 7),
    ("DGS10", "10Y", 10),
    ("DGS20", "20Y", 20),
    ("DGS30", "30Y", 30),
]

OECD_10Y_SERIES = [
    ("IRLTLT01USM156N", "United States"),
    ("IRLTLT01DEM156N", "Germany"),
    ("IRLTLT01FRM156N", "France"),
    ("IRLTLT01ITM156N", "Italy"),
    ("IRLTLT01GBM156N", "United Kingdom"),
    ("IRLTLT01JPM156N", "Japan"),
    ("IRLTLT01ESM156N", "Spain"),
    ("IRLTLT01PTM156N", "Portugal"),
    ("IRLTLT01GRM156N", "Greece"),
]

ALL_FRED_IDS = tuple(s for s, _, _ in TREASURY_SERIES) + tuple(
    s for s, _ in OECD_10Y_SERIES
)

DEFAULT_TREASURY_TENORS = ("3M", "2Y", "10Y", "30Y")
FRED_EARLIEST = date(1953, 4, 1)

```

Helpers:

```
def _latest_snapshot(frame: pd.DataFrame):
    filled = frame.ffill()
    valid = filled.dropna(how="all")
    if valid.empty:
        return None
    as_of = valid.index[-1]
    return as_of, filled.loc[as_of]

def _earliest_yield_date() -> date:
    starts = [FRED_EARLIEST]
    for series_id in ALL_FRED_IDS:
        bounds = get_available_date_bounds(series_id)
        if bounds is not None:
            starts.append(bounds[0])
    return min(starts)
```

Sidebar and download

Same lookback pattern as Stocks, but keys `bonds_lookback`, `bonds_start_date`, `bonds_end_date`.

Add two multiselects: OECD country **labels** (default all) and Treasury **tenor** labels (default 3M, 2Y, 10Y, 30Y). They filter **history** charts only. The latest Treasury **curve** still uses every tenor that has a value.

```
yields = download_fred_data(ALL_FRED_IDS, start_date, end_date, use_live=True)
if not yields.empty:
    yields = yields.ffill().dropna(how="all")
```

Four charts

Follow the projector. Sketch:

- **OECD 10Y snapshot** — bar chart of the latest yield per selected country (`sort_values`).
- **OECD 10Y history** — one line per selected country.
- **US Treasury curve snapshot** — scatter+lines of tenor vs yield (sort by years).
- **US Treasury history** — lines for selected tenors only.

Use `chart_layout(...)` and `st.plotly_chart(..., use_container_width=True)`.

If the room is short on time: keep **one** snapshot and **one** history. Do not start Commodities until four charts work **or** the instructor says to stop.

Commit

```
git add views/common.py views/bonds.py
git commit -m "session 10: bonds"
```

Next week

Stretch: Commodities, then Currencies. If Bonds is still broken, **fix Bonds** — that is valid work. Session 12 is deploy, not new features.